通用神经网络处理器下的多核调度问题

随着人工智能模型规模持续增长，单个 AI 核心受制造成本、功耗与散热约束，难以独立满足算力需求，多核 NPU 因而成为提升并行能力与能效的主流架构。多核调度需要将完整计算图划分为子图并分配到多个核心，在利用核内高效计算单元与私有缓存复用的同时协调跨核依赖。其关键是在多核并行度、核内缓存容量以及共享主存的数据搬运带宽之间权衡，减少数据传输与资源争用并缩短总体任务执行时间。

1. 多核调度问题

1.1 多核 NPU 平台

考虑由 N 个同构 AI 核心与共享核外主存（后续简写为 DDR，双倍数据率动态随机存取内存）构成的多核 NPU：每核配置 Cube/Vector 计算单元、数据搬运单元及私有 L1/UB 缓存。所有核心对 DDR 内数据的读取与写入共同竞争固定的总物理带宽。完整硬件结构与图示见附录 A。

1.2 计算图与符号定义

设多核平台包含 N 个同构 NPU 核心。待调度对象是一张有向无环计算图（DAG，Directed Acyclic Graph）。多核调度算法需要将该计算图划分为若干子图，将子图分配到各核心，并确定每个核心上的子图执行顺序；在满足数据依赖、缓存容量、计算流水线和共享带宽等约束的前提下，减少核外数据搬运、提高多核并行度，最终缩短总体任务执行时间。

实际神经网络的计算数据通常远大于单核缓存容量，因而现实场景需要将各种计算拆解为可在核内执行的细粒度操作和数据块。本题给出的计算图用于描述这类分块计算形成的数据流：操作节点表示计算或搬运，张量节点表示其输入、中间结果和输出。以分块矩阵乘法为例，每个数据分块依次经历“从 DDR 搬入 L1 → 由 Cube 计算 → 将结果写回 DDR”，由此形成图 1 所示的计算图；图中箭头表示节点之间的数据依赖。

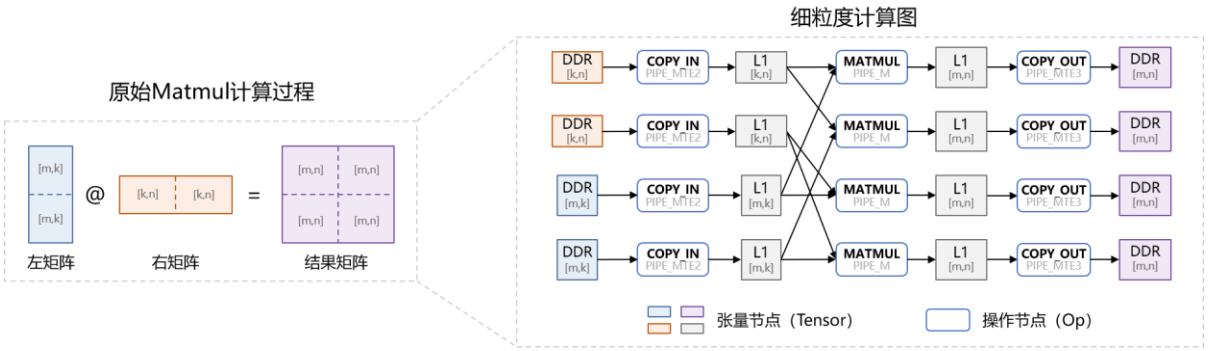


图 1　矩阵乘法的示例计算图

形式上，将计算图记为 $G = (V, E)$。其中，张量节点集合为 T，操作节点集合为 O，二者满足 $T \cap O = \emptyset$；全部节点的集合为 $V = T \cup O$，有向边集合为 E。每个节点都有唯一的整数标识符 id。计算图的输入、输出张量位于 DDR；中间张量的逻辑位置为核内缓存（L1 或 UB）。

张量节点（Tensor）表示一块数据，记作 $t \in T$，可作为计算图输入、输出或中间结果；每个节点包含以下属性：

- **位置：**表示输入图规定的逻辑存储位置，可取 DDR、L1 或 UB。DDR 是核外主存；L1 和 UB 是每个核心私有的缓存，分别主要服务于 Cube 与 Vector 计算，具体位置以 pos 字段为准。计算图的输入、输出张量必须位于 DDR。
- **数据大小：**张量 t 占用的字节数，记为 s_t。该值为非负整数，同时用于核内容量检查和数据搬运量计算。

操作节点（Op）表示一次计算或数据搬运，记作 $o \in O$。COPY_IN 将数据从 DDR 搬入 L1/UB，COPY_OUT 将数据从 L1/UB 搬回 DDR；ADD、MUL 等核内计算操作在指定的计算单元上执行。每个节点包含以下属性：

- **操作类型：**标识具体操作，例如 COPY_IN、COPY_OUT、ADD 或 MUL；不要与操作节点集合 O 混淆。
- **执行管道：**指定操作占用的硬件流水线。PIPE_M 和 PIPE_V 分别对应 Cube 与 Vector 计算单元；PIPE_MTE2 负责 DDR → L1/UB 以及 UB → L1，PIPE_MTE3 负责 L1/UB → DDR 以及 L1 → UB。不同 Pipe 能否并行取决于数据依赖和资源约束。
- **执行时间：**对核内计算操作 o，其执行周期数记为 c_o，由计算图给定。访问 DDR 的搬运操作不按固定周期数结算，其最终时间由数据量和评估时刻的可用 DDR 带宽共同决定，见 1.4 节。

有向边（Edge）集合满足 $E \subseteq (T \times O) \cup (O \times T)$，只允许张量节点与操作节点相连。对 $t \in T$、$o \in O$，$t \to o$ 表示 t 是 o 的输入，$o \to t$ 表示 t 是 o 的输出；不允许 Tensor→Tensor 或 Op→Op。边本身不设置通信量：当依赖跨越子图或执行任务（Task）时，搬运字节数由关联张量的大小 s_t 决定。Task 指 CPU 主控向单个 NPU 核心一次下发并统一调度的最小执行单元；一个 Task 可包含一个或多个子图，具体对应关系由场景决定，见 1.5 节。

1.3 问题目标

本题要求提出高效且效果良好的多核切图与调度方法：在给定计算图和核心数量后，能够在合理时间内生成符合要求、可执行的多核切分与调度方案，并尽可能缩短总体任务执行时间（Makespan）、控制额外数据搬运。为此，方法需要联合确定各操作节点的子图归属、子图的核心分配以及各核心上的子图执行顺序。

合理的调度方案能够把相互独立或资源不同的计算分散到不同核心并行执行，同时尽量保留计算过程中数据的核内复用，从而提高计算单元利用率、减轻 DDR 带宽压力。因此，本问题不仅决定多核算力能否得到有效利用，也直接影响任务时延、数据搬运开销和系统能效。

求解困难来自三个相互耦合的决策：切图、核心的分配和同一核内的子图调度顺序。其中，切图改变子图规模、并行空间和通信边界；核心分配决定负载均衡及跨核通信；同一核内的子图调度顺序影响依赖等待、张量驻留时间和缓存换入换出。减少切分可降低通信，却可能压缩并行度并增大缓存压力；增加切分可提供更多并行机会，却可能带来重复读取和更多 DDR 搬运。因此，不能孤立优化任一环节，而需在数据依赖、流水线、缓存和带宽约束下进行联合权衡。

多核调度按两个相互衔接的过程展开：（1）切图，将完整计算图划分为若干子图，子图数量不作限制；（2）子图调度，将各子图分配到具体核心，并确定每个核心上的子图执行顺序。算法需输出**切图方案**和**子图调度方案**；赛题提供的评估程序（参见附录 B）会根据算法结果，补充跨子图边界数据搬运、确定核内操作顺序，并模拟多核在共享带宽下的并行执行，最终给出各项评估指标。

切图方案：以计算图中每个**核内操作节点**至其子图 id（Subgraph id，下文简称 sgid）的映射表来表示（核内操作节点为除 COPY_IN/COPY_OUT 外所有节点）。赛题所提供的评估程序根据原始计算图和切图结果自动重建边界数据搬运（如图 2 所示）。映射表必须包含所有核内操作节点，且由跨子图依赖形成的子图依赖图必须为 DAG（至少存在一个合法拓扑序），否则方案无效。

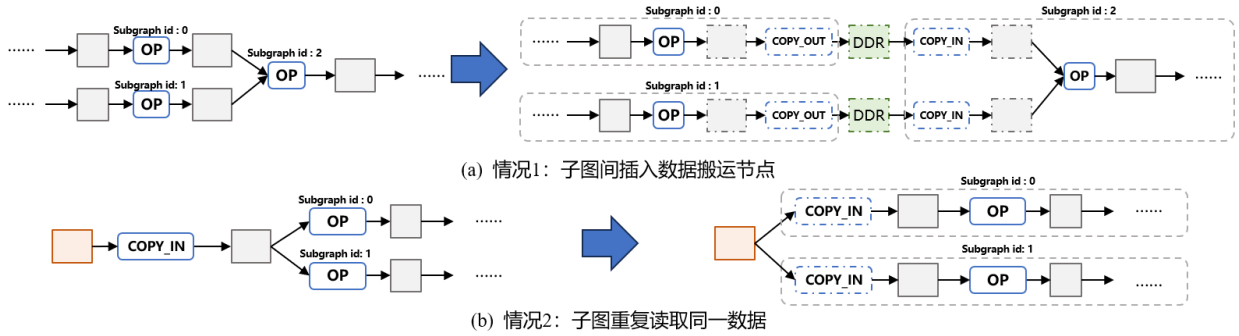


图 2　切图后数据搬运节点插入规则

子图调度方案：以 *N* 个 sgid 列表表示：每个列表表示一个 NPU 核心上的子图执行顺序。每个 sgid 必须且只能出现一次，且同核内的子图调度顺序不得违反子图间依赖关系。

1.4 评估指标

总体任务执行时间（Makespan）：最重要指标，由赛题提供的评估程序统一计算，单位为时钟周期。该评估程序分为构建子图间数据通路、确定每个子图核内调度顺序、多核模拟执行给出指标三个阶段：首先，按照 1.5 节的场景规则构造 Task 图、补充跨 Task 的边界搬运；随后，采用附录 C 的核内调度算法，确定每个 Task 中操作的固定执行顺序；当 L1/UB 容量不足时，该阶段会自动插入缓存换出和缓存换入。最后，多核模拟执行在遵循上述核内操作顺序和方案给出的子图间调度顺序的基础上，模拟所有核心的并行执行。评估程序结束时给出所有操作结束时刻的最大值为 Makespan，同时给出其余评估指标。该多核模拟过程考虑多核共享 DDR 带宽争用，具体算法见附录 D。

访问 DDR 的 COPY_IN、COPY_OUT、缓存换出和缓存换入共享固定 DDR 总带宽。核内调度阶段使用的可配置带宽只用于确定核内操作顺序；最终 Makespan 始终按固定总带宽及并发搬运争用重新计算。

总额外数据搬运量：次要指标，表示多核调度相对于原始计算图新增的 DDR 搬运字节数，即调度后的全部 DDR 搬运量扣除原始计算图已有的搬运量。

新增搬运主要来自三类情况：跨越必须经 DDR 中转的 Task 边界数据、多个 Task 对同一输入的重复读取，以及核内调度中 L1/UB 容量不足时自动产生的缓存换出与后续缓存换入。

哪些子图间依赖需要经 DDR 中转，取决于 1.5 节定义的 Task 组织与通信方式；赛题提供的评估程序按对应规则自动重建搬运并统计字节数。

该指标用于衡量切图、子图分核和核内缓存复用对 DDR 流量的影响。

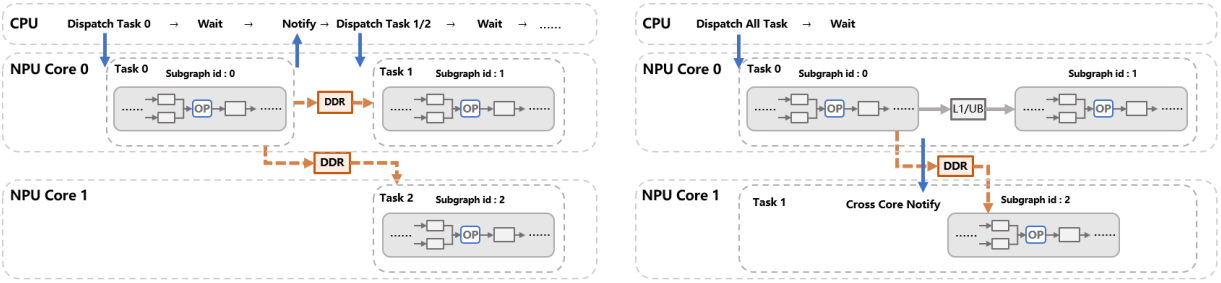
Cache 命中率：次要指标，表示可由只读 Cache 服务的 COPY_IN 访问中，命中字节数占总访问字节数的比例。该指标用于观察 Cache 对共享输入访问的覆盖程度。

1.5 硬件场景

根据硬件平台是否具有核间同步信号传递机制，多核调度分为两种硬件场景：

场景 A：无核间同步机制的场景。一个 Task 只能包含一个子图，子图间的同步依赖由 CPU 主控通过 Task 调度完成。同一核上多个 Task 串行执行时，核内缓存状态会被清空，因此所有跨子图数据必须经 DDR 中转；任意两子图间的单位通信成本相同（$C_{\text{inner}} = C_{\text{cross}}$），且跨 Task 数据不驻留私有缓存。

场景 B：存在核间同步机制的场景。一个 Task 可包含同一核心上的多个子图，前序子图的数据可驻留在 L1/UB 中供后续子图直接使用。跨核依赖由源核心将数据写入 DDR 并发送同步信号，目标核心收到信号后再从 DDR 读取。因此，同核单位通信成本 C_{inner} 小于跨核单位通信成本 C_{cross}（$C_{\text{inner}} < C_{\text{cross}}$）。但跨越多个子图的驻留数据会持续占用核内缓存，可能压缩中间子图的可用空间并降低执行效率。



(a) 场景 A：核间同步需通过 Task 编排完成 / 子图间只允许 DDR 数据通信　(b) 场景 B：允许 Task 内部传递核间同步信号 / 同核子图间允许核内数据通信

图 3　两种场景下子图间通信方式对比

因此，边界数据搬运节点只为跨 Task 通信插入；场景 B 中同一 Task 内的子图通信不插入数据搬运节点。

在上述两个场景的基础上，进一步引入所有核心共享的二级缓存 L2（Level 2 Cache）。L2 采用只读 Cache 模式，容量为 1 MB、带宽为 250 bytes/cycle，用于复用多核共享输入；L2 带宽与 DDR 带宽相互独立、互不占用。

问题 1：场景 A 下的多核切图与调度算法

场景 A 下，每个子图独立构成一个 Task，所有跨子图数据均须经 DDR 中转。参赛者需建立多核切图与调度模型并设计求解算法，为每个非 COPY_IN/COPY_OUT 操作分配子图 id，同时确定各核心上的子图执行顺序。方案以缩短总体任务执行时间为主要目标，并兼顾总额外数据搬运量；算法还应适应核数变化，在 2 ~ 5 核范围内均能稳定地、高效地[1]生成高质量结果。

参赛者须通过代码实现所提出的算法，在题目给定的全部测试用例和固定配置参数下完成实验，并提交以下三类结果：

- **正文：**给出 1 ~ 5 个核的平均加速比折线图[2]，作为关键算法结果。单核基准 Makespan 由核内调度算法在整图上计算，单核加速比定义为 1[3]；对于 k 核配置，加速比定义为单核配置下核内调度所得 Makespan 与 k 核下多核评估所得 Makespan 之比。
- **附录：**逐用例列出赛题评估程序给出的结果，包括 Makespan 和总额外数据搬运量。
- **附件：**提供可复现的 Python 程序脚本，输入输出格式见附录 B。

问题 2：场景 B 下的多核切图与调度算法

场景 B 下，每个核心上的全部子图合并为一个 Task：同核子图可直接复用 L1/UB 中的数据，跨核依赖则通过 COPY_OUT/COPY_IN 和固定同步延迟完成。参赛者需在问题 1 的基础上，建立适用于场景 B 的切图与调度模型并设计求解算法，在严格满足 L1/UB 容量约束的前提下，进一步缩短总体任务执行时间并减少额外数据搬运。算法应适应核数变化，在 2 ~ 5 核范围内均能稳定地、高效地生成高质量结果。

参赛者须在题目给定的全部测试用例和固定配置参数下完成实验，并提交以下三类结果：

- **正文：**给出场景 B 下 1 ~ 5 个核的平均加速比折线图，加速比定义与问题 1 相同。
- **附录：**逐个用例地列出赛题评估程序给出的结果，包括 Makespan 和总额外数据搬运量。
- **附件：**提供可复现的 Python 程序脚本，输入输出格式见附录 B。

问题 3：共享 L2 资源下的多核切图与调度

本问题在场景 B 的基础上引入所有核心共享的只读 L2 Cache，要求参赛者研究其容量和带宽对多核切图与调度的影响，并建立相应模型、设计求解算法。L2 容量为 1 MB、带宽为 250

---

[1] 此处“高效地”代表着算法应该避免使用暴力迭代搜索机制，以单个用例能在 5 ~ 10 分钟内给出结果为佳。

[2] 对于平均加速比，统一定义为所有用例的加速比的平均值，而非所有用例上的单核总时间除以多核总时间。

[3] 请统一使用启发式核内调度评估得到的固定结果作为单核基准。由于启发式算法无法给出最优调度结果，因此可能出现多核加速比大于核数的正常情况。

bytes/cycle，用于复用多核共享输入，其带宽独立于 DDR 带宽。算法应适应核数变化，并能稳定地、高效地生成高质量结果。

参赛者须在题目给定的全部测试用例和固定配置参数下完成实验，并提交以下两类结果：

- **正文：**给出 1 ~ 5 核下无 L2 与只读 Cache 两种配置的对比曲线，并报告相同核数下只读 Cache 相对无 L2 基线的加速比；该加速比定义为无 L2 时的 Makespan 与只读 Cache 配置下 Makespan 之比。
- **附录：**逐用例列出无 L2 与只读 Cache 两种配置的赛题评估程序给出的结果，包括 Makespan、总额外数据搬运量、Cache 命中率。

算法优化方向

如下优化方向供参考：

子图间通信成本：如切图方案部分所述，跨子图边的通信数据量需要视场景插入额外的数据搬运节点，从而增加对 DDR 共享带宽的消耗；强通信依赖的两子图若被合并为一个子图，可显著降低通信开销，但会牺牲并行度。

多核并行度：核数 N 固定时，子图划分方式与子图调度方案共同决定各核负载均衡程度与并行度上限；过度聚合子图会压缩多核并行空间，过度切分则会放大跨子图通信代价。

数据重复读取代价：同一份数据可能被多个计算节点使用；如果这些节点被切分至多个子图中，则每份子图均需从核外主存 DDR（或 L2 Cache）各自读取该数据，从而显著增加对共享 DDR 带宽的消耗。切图与子图调度应尽量将共享同一输入数据的节点聚合到同一子图（或同一核），以减少重复读取带宽代价。

单核内矩阵单元与向量单元的自动并行：同一 AI 核心内的 Cube 与 Vector 单元可同时独立执行；因此，在切图阶段需考虑同一子图内两个计算单元之间的负载均衡，以提升核心整体执行效率、避免某一计算单元长期空闲。

核内启发式调度对大型计算图处理效率的影响：现有核内调度算法为启发式算法（完整代码在附件中给出，并在附录 C 中说明具体原理）。当单个子图规模过大时，核内调度算法需引入额外的数据搬运（缓存换入换出）以缓解核内 L1/UB 容量压力。在超大规模子图上，启发式调度的非全局最优解，可能导致大量的中间数据被换入换出，由此带来额外的数据搬运最终导致核内调度效率下降。一般情况下，两个子图合并后的子图执行时间，会小于两子图各自单独执行的时间之和（时间收益来自于合并后前后子图头尾数据搬运开销被计算流水掩盖，以及通信成本的降低）。但当子图合并后的规模超过一定程度时，可能出现合并后负时间收益的现象。

单核内通信对缓存资源的占用：在场景 B 中，同一核心上的多个子图可以通过核内缓存复用驻留数据，减少切分子图引入的数据搬运。然而，子图间的数据驻留会持续占用核内缓存，压缩其他子图执行时可用的缓存空间，从而可能降低单个子图的调度效率。

附录 A 多核硬件架构

平台由 N 个同构 AI 核心和核外主存 DDR 组成，各核心共享 DDR 核外主存。COPY_IN、COPY_OUT、缓存换入和缓存换出均计入 config.txt 规定的 DDR 总带宽（本题固定为 60 bytes/cycle），该带宽被读写操作共享。

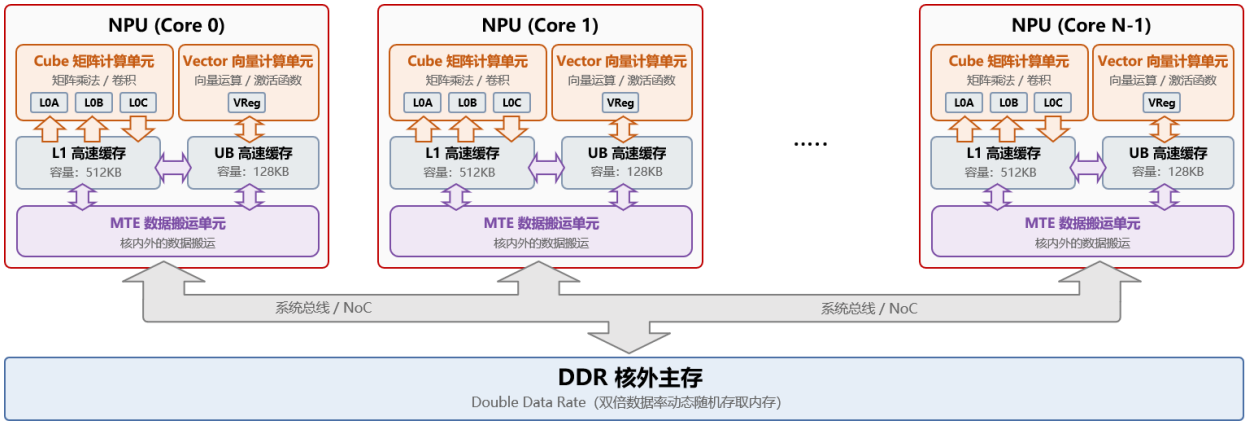


图 4　多核 NPU 平台硬件架构

每个核心包含 Cube 矩阵计算单元（PIPE_M）、Vector 向量计算单元（PIPE_V）、两类搬运单元（PIPE_MTE2、PIPE_MTE3），以及私有 L1、UB 缓存（图中 L0A、L0B、L0C、VReg 缓存不用在本题中考虑）。PIPE_MTE2 支持 DDR → L1/UB 和 UB → L1，PIPE_MTE3 支持 L1/UB → DDR 和 L1 → UB。Cube 与 Vector 可并行执行，但仍受数据依赖和各自 Pipe 占用约束。

附录 B 附件内容、数据格式与使用说明

本附录说明赛题材料包的目录、脚本职责、计算图与方案格式，以及从生成方案到完成问题 1~3 评估的示例流程。算法只需按照规定格式输出多核调度方案；赛题提供的评估程序在读取该方案后，统一评估并输出 Makespan、搬运量及 Cache 指标。

B.1 材料包目录

- **README.md：**材料包总览、固定配置和常用命令。
- **code/：**问题 1~3 的评估程序，均以 Python 脚本形式提供；目录额外包含辅助功能脚本（核内调度脚本和日志脚本）。
- **data/：**正式测试计算图及统一配置 config.txt。论文中的算法评估结果必须由使用该配置的评估脚本给出，不得修改。
- **docs/：**核内调度算法和多核并行模拟算法的补充说明。

B.2 脚本功能

三个问题的评估脚本如下：

- **multicore_cut_evaluate_problem_1.py：**输入计算图路径和多核调度结果，给出问题 1 的评估结果。
- **multicore_cut_evaluate_problem_2.py：**输入计算图路径和多核调度结果，给出问题 2 的评估结果。
- **multicore_cut_evaluate_problem_3.py：**输入计算图路径和多核调度结果，给出问题 3 的评估结果。

其他为辅助脚本（核内调度评估脚本等）。

- **schedule_step1.py：**将 Op-Tensor 二部图转换为操作依赖图，生成确定性的核内拓扑执行顺序。
- **schedule_step2.py：**在给定 L1/UB 容量下检查张量驻留，并在容量不足时插入缓存换出和缓存换入。
- **schedule_step3.py：**在四条 Pipe 上按依赖和资源约束排布扩展执行图，形成核内时间线；其结果用于确定顺序，不直接作为最终多核 Makespan。
- **stub_multicore_cut_and_schedule.py：**读取计算图，随机生成格式合法的多核调度结果文件。该脚本只演示接口与校验流程，不是性能基线。

B.3 原始计算图 JSON

计算图 json 文件采用 UTF-8 编码，顶层包含 tensors、ops 和 edges 三个数组。所有节点 id 在同一计算图内全局唯一，数值字段为整数且不得为负。

- **tensors：**每项包含 id、pos、size。pos 可取 DDR、L1 或 UB；size 表示字节数。计算图输入、输出张量位于 DDR，中间张量位于 L1 或 UB。
- **ops：**每项包含 id、op、pipe、cycles。COPY_IN 和 COPY_OUT 分别表示 DDR 到核内、核内到 DDR 的搬运；其他操作在 PIPE_M 或 PIPE_V 上执行。
- **edges：**每项包含 source 和 target，只允许 Tensor → Op 或 Op → Tensor，用于表示输入、输出依赖。边本身不重复存储数据量，搬运字节数由相连张量的 size 决定。

本题提供的计算图保证每个 DDR 输入只搬入一次、每个最终结果只搬出一次；中间计算不与 DDR 交换数据；单个计算操作的输入、输出总量不超过相应核内缓存容量。

B.4 config.txt 配置说明

config.txt 与计算图默认放在同一目录，也可通过 --config 显式指定。固定配置如下：L1 为 524288 bytes，UB 为 131072 bytes，共享 DDR 总带宽为 60 bytes/cycle；问题 1 的跨核前驱等待和同核 Task 切换等待分别为 1000、100 cycles；问题 2、3 的跨核拷贝同步延迟为 500 cycles；问题 3 的只读 Cache 为 1048576 bytes、250 bytes/cycle。

B.5 多核方案输出格式

参赛算法输出 **<case>_multicore_res.json**，其中 <case> 表示用例名称，即输入计算图文件去掉 .json 后的文件名。方案文件顶层必须且只能包含以下两个字段：

- **node_to_subgraph**：对象键为非 COPY_IN/COPY_OUT 操作 id 的十进制字符串，值为非负整数 sgid；必须恰好覆盖全部非 COPY 操作。
- **core_schedules**：二维整数数组。外层下标为核心 id，内层数组按执行顺序列出该核的 sgid，可为空；全部内层数组合并后，每个 sgid 必须且只能出现一次。

评估器还会检查子图依赖图无环，以及同核调度顺序不违反依赖。不同文本键若转换为同一整数 id，也按重复 id 处理。

B.6 输入、运行与输出

以下命令均在材料包根目录下执行。先由参赛算法生成方案；如只需验证格式，可运行示例生成器：

```
python code/stub_multicore_cut_and_schedule.py data/<case>.json -n 4
```

该命令默认在输入计算图所在目录生成 **<case>_multicore_res.json**；例如输入 data/<case>.json 时，产物为 data/<case>_multicore_res.json。该文件即后续评估器读取的多核切图与调度方案。

随后使用同一份方案依次评估问题 1、问题 2 和问题 3 的只读 Cache 配置：

```
python code/multicore_cut_evaluate_problem_1.py data/<case>.json --config data/config.txt
python code/multicore_cut_evaluate_problem_2.py data/<case>.json --config data/config.txt
python code/multicore_cut_evaluate_problem_3.py data/<case>.json --config data/config.txt
```

评估命令的第二个位置参数为方案文件。若省略该参数，评估器会在输入计算图所在目录查找同名的 **<case>_multicore_res.json**；若文件不存在，则报错并终止评估，不会自动生成方案。config.txt 同样默认从输入图目录读取，也可通过 --config 显式指定；-o、--trace-output、--log-output 分别用于指定三类输出路径。

上述每个评估脚本均生成以下三类输出文件。文件名中的 <problem> 表示评估类型：problem_1、problem_2 和 problem_3 分别对应问题 1、问题 2 和问题 3：

- **结果 JSON**：默认文件名为 <case>_<problem>_res.json，是数值评估与自动检查的输出。
- **Trace JSON**：默认文件名为 <case>_<problem>_trace.json。文件采用 Perfetto 可直接读取的 Trace Event JSON（Chrome JSON Trace）格式；在浏览器打开 https://ui.perfetto.dev 后，点击 Open trace file 或直接拖入文件即可查看。
- **精简日志**：默认文件名为 <case>_<problem>_log.txt，用于快速核对配置、Makespan、搬运量、缓存峰值及 Task/子图时间段。

B.7 最小输入输出示例

以下计算图包含一次 DDR→UB 搬入、一次向量 ADD 和一次 UB→DDR 搬出：

```
{
  "tensors": [
    {"id":1, "pos":"DDR", "size":16},
    {"id":2, "pos":"UB",  "size":16},
    {"id":3, "pos":"UB",  "size":16},
    {"id":4, "pos":"DDR", "size":16}
  ],
  "ops": [
    {"id":10, "op":"COPY_IN",  "pipe":"PIPE_MTE2", "cycles":1},
    {"id":11, "op":"ADD",      "pipe":"PIPE_V",    "cycles":4},
    {"id":12, "op":"COPY_OUT", "pipe":"PIPE_MTE3", "cycles":1}
  ],
  "edges": [
    {"source":1,"target":10}, {"source":10,"target":2},
    {"source":2,"target":11}, {"source":11,"target":3},
    {"source":3,"target":12}, {"source":12,"target":4}
  ]
}
```

对应的两核调度方案只需把非 COPY 操作 11 放入子图 0，并调度到核心 0：

```
{
  "node_to_subgraph": {"11": 0},
  "core_schedules": [[0], []]
}
```

将上述计算图和调度方案以文本的形式保存成两个文件（UTF-8 编码），并分别命名为 minimal_graph.json 和 minimal_graph_multicore_res.json 后，执行：

```
python code/multicore_cut_evaluate_problem_1.py minimal_graph.json --config data/config.txt
python code/multicore_cut_evaluate_problem_2.py minimal_graph.json --config data/config.txt
python code/multicore_cut_evaluate_problem_3.py minimal_graph.json --config data/config.txt
```

在固定配置下，三次评估均得到 Makespan = 6 cycles；原图和调度后 COPY 总量均为 32 bytes，新增搬运量为 0。该图没有可复用数据，因此只读 Cache 的命中次数为 0。结果 JSON 的关键字段示例如下：

```
{
  "scene": "A",
  "makespan": 6,
  "num_cores": 2,
  "data_movement_bytes": {
    "original_graph_copy_bytes": 32,
    "scheduled_copy_bytes": 32,
    "added_copy_bytes": 0
  }
}
```

附录 C 核内调度算法

核内调度以单个 Task 的计算图及 L1/UB 容量、带宽参数为输入，依次生成确定性拓扑序、插入必要的缓存换入换出操作，并完成多 Pipe 排布。输出包括扩展图、确定性操作顺序和各 COPY 独占带宽时的参考工作量，供附录 D 的多核模拟使用；本阶段的局部时间仅用于排布，不作为最终 Makespan。详细步骤、伪代码和边界条件见附件 **docs/核内调度算法.md**。

C.1 步骤 1：确定性拓扑排序

算法先将 Op-Tensor 二部图转换为 Op-Op 依赖图，再从无后继操作节点开始反向深度优先搜索（DFS）；仅当全部前驱均已处理时，操作才进入序列。并列候选按操作类型、预计算深度和 id 的固定优先级排序，使 COPY_IN 尽量靠近使用点，并保证结果可复现。对问题 2、3 的合并 Task，算法还在不破坏依赖的前提下按 core_schedules 调整子图间顺序，并再次校验拓扑合法性。**对应脚本：code/schedule_step1.py。**

C.2 步骤 2：缓存容量约束与换入换出

算法沿步骤 1 的固定顺序维护 L1/UB 驻留张量及其下一次使用位置。当为当前操作申请空间会超出容量时，从不属于该操作输入或输出、且未来仍会使用的驻留张量中，选择下一次使用最晚者换出。若 DDR 中没有有效副本，先插入 COPY_OUT 写入持久后备区；再在该张量下次使用前插入 COPY_IN，并更新张量引用和空间复用依赖。算法重复换出直至容量满足；若单个张量超过容量或不存在合法换出对象，则该 Task 失败。 **对应脚本：code/schedule_step2.py。**

C.3 步骤 3：多 Pipe 乱序流水排布

在核内排布阶段，操作在依赖满足、对应 Pipe 空闲且容量允许时尽早发射。PIPE_M、PIPE_V、PIPE_MTE2 和 PIPE_MTE3 各自串行，不同 Pipe 可并行。计算操作的耗时取给定周期数；COPY 仅记录其独占对应带宽时的参考工作量，实际完成时间由附录 D 按共享带宽重算。产生核内缓存输出的操作须按扩展序列申请空间，输入张量在最后一次使用完成后释放。**对应脚本：code/schedule_step3.py。**

C.4 输出与算法边界

输出包括扩展后的 ops、tensors、edges、确定性操作序列、L1/UB 峰值和新增换入换出量。核内阶段不处理跨核同步、共享 DDR/Cache 带宽或全局时间，这些均由附录 D 统一模拟。

附录 D 多核并行模拟执行过程

评估程序首先读取原始计算图和算法给出的多核切图与调度方案，随后构造 Task 及跨核依赖，再调用附录 C 中核内调度算法来确定每个 Task 内部节点执行顺序与内部执行依赖，最后在统一事件时间轴上模拟各核心、Pipe、核内存储和共享带宽。下文按评估器的执行顺序仅做简要说明，详细步骤与伪代码见附件 **docs/多核并行模拟执行算法.md**。

D.1 输入与方案合法性检查

输入包括原始计算图、node_to_subgraph、core_schedules 和 config.txt。评估器据此确定每个非 COPY 操作所属的子图、每个子图所属的核心及同核执行顺序，并检查：节点覆盖且无重复，sgid 为非负整数，每个子图恰好调度一次，子图依赖无环，同核顺序不违反依赖。任一检查失败即终止评估。

D.2 Task 与跨核依赖构造

问题 1：每个子图构成一个独立 Task，并按 core_schedules 分配到核心。Task 之间不保留核内数据：所有边界数据均经 DDR 中转，并由 COPY_OUT/COPY_IN 传递，即使相邻 Task 位于同一核心也是如此。Task 的最早激活时刻同时受同核顺序和前驱约束：同核前一 Task 完成后至少等待 100 cycles；每个跨核前驱 Task 完成后至少等待 1000 cycles。实际激活时刻取全部约束的最大值。

问题 2、3：同一核心的全部子图合并为一个 Task；同核依赖保留核内数据，不插入边界 COPY。仅对跨核数据边，在源核插入 COPY_OUT、目标核插入 COPY_IN。目标 COPY_IN 须在源 COPY_OUT 完成并经过 500 cycles 同步延迟后，才能进入就绪判定。问题 3 沿用问题 2 的 Task 结构，仅增加只读 Cache 资源。

D.3 核内展开与模拟状态初始化

每个 Task 按附录 C 展开，得到包含必要换入换出操作的扩展图和确定性操作顺序。随后为每个核心初始化四条 Pipe 的就绪队列与执行器，以及操作状态、前驱计数、L1/UB 驻留集合、剩余引用、当前占用和峰值；全局初始化 DDR 带宽池，问题 3 另初始化只读 Cache 及 Cache 读带宽池。

D.4 就绪判定、存储约束与操作发射

操作状态依次为 pending、ready、running 和 done。进入 ready 必须同时满足：所属 Task 已激活，全部核内前驱已完成，跨核释放时刻已到；若操作需要申请核内缓存，还须轮到其申请次序且 L1/UB 容量足够。就绪操作按附录 C 给出的确定性顺序进入本核对应 Pipe 的队列；Pipe 空闲时发射，同核同 Pipe 串行，不同 Pipe 或不同核心可并行。发射前申请输出空间，操作完成后递减输入张量引用，最后一次使用完成即释放空间。

D.5 共享带宽与只读 FIFO Cache

所有需要访问 DDR 的搬运（问题 3 中即 Cache 未命中的搬运）共享 60 bytes/cycle 总带宽。每个在途搬运以“独占全部 DDR 带宽时尚需的周期数”记录剩余工作量。新搬运加入或已有搬运完成时，先按原并发数结算上一时段，再由当前搬运公平共享带宽，并重算预计完成时刻。L1/UB 之间的核内 COPY 操作只占对应 Pipe，不进入 DDR 带宽池。

问题 3 在 COPY_IN 发射时按逻辑 tensor id 查询只读 FIFO Cache。命中不改变 FIFO 顺序，并进入 Cache 读带宽池；未命中则访问 DDR，完成后写入 Cache，容量不足时按先进先出淘汰。Cache 读与 DDR 流量隔离，两个带宽池分别按上述公平共享规则结算，互不占用带宽；单个 tensor 大于 Cache 容量时不缓存。

D.6 离散事件循环与时间推进

设当前时刻为 now，每轮严格按以下顺序执行：

- **结算完成事件：**更新各带宽池的剩余工作量，将结束时刻不晚于 now 的操作置为 done，并释放对应 Pipe。

- **更新依赖与存储：**递减后继的前驱计数和输入张量的剩余引用，释放最后一次使用完成的核内张量；问题 3 同时处理 Cache 插入和 FIFO 淘汰。
- **激活 Task 与跨核释放：**按同核顺序和前驱条件激活 Task；对附加延迟已到期的跨核 COPY_IN 解除等待，使其进入就绪判定。
- **发射就绪操作：**逐核、逐 Pipe 发射满足依赖、执行器、申请次序和容量条件的操作；带宽操作加入相应共享池，并立即重算预计完成时刻。
- **推进时间：**取运行中操作的预计完成时刻与待处理跨核释放时刻中的最小值作为下一时刻，进入下一轮。